Modeling Attack Resistant Arbiter PUF with Time-Variant Obfuscation Scheme

Zhengtai Chang
School of Information Engineering
Zhengzhou University
Zhengzhou, China
changzhengtai@163.com

Shanshan Shi
School of Information Engineering
Zhengzhou University
Zhengzhou, China
shishanshan2020@163.com

Binwei Song
School of Information Engineering
Zhengzhou University
Zhengzhou, China
songbinwei2021@163.com

Wenbing Fan
School of Information Engineering
Zhengzhou University
Zhengzhou, China
iewbfan@zzu.edu.cn

Yao Wang*
School of Information Engineering
Zhengzhou University
Zhengzhou, China
ieyaowang@zzu.edu.cn

2021 31st International Conference on Field-Programmable Logic and Applications (FPL) | 978-1-6654-3759-2/21/$31.00 ©2021 IEEE | DOI: 10.1109/FPL53798.2021.00018

***Abstract*—Strong PUF represented by arbiter PUF is suitable for the authentication of resource-constrained devices. However, conventional arbiter PUF is vulnerable to modeling attacks due to its linear structure. In this paper, we propose an arbiter PUF with time-variant obfuscation scheme (TVO-APUF), which feeds the external random challenges into the linear feedback shift register (LFSR) module to determine the real challenge of underlying arbiter PUF, thus obfuscating the linear mapping relationship between challenge and response, leading to significant resistance to modeling attacks. In addition, LFSR module with low hardware overhead can be updated at any time to prevent reply attack. We implement a 48-stage TVO-APUF on Xilinx Spartan-6 FPGA board. The experimental results show that the proposed TVO-APUF can effectively resist modeling attacks such as logistic regression (LR), support vector machine (SVM) and evolutionary strategy (ES) with a maximum prediction rate of 53% and slight effects on uniformity, stability and uniqueness.**

Keywords—hardware security; arbiter PUF; modeling attack; time-variant obfuscation; LFSR.

I. Introduction

With the rapid development of information technology, information security has been considered in most applications. These problems can be addressed by various encryption algorithms such as AES and RSA [1]. The secret key, critical for the encryption algorithm and should remain confidential, is commonly restored in non-volatile memory (NVM). However, some devices are highly resource-constrained, making it difficult to implement complex encryption algorithms. Furthermore, the NVM restoring the secret key is vulnerable to various physical attacks, including side-channel, semi-invasive, and invasive attacks [2].

Physical unclonable function (PUF) represents a promising hardware security primitive for resource-constrained devices [3]. PUF employs the inevitable random variations in chip manufacturing to extract the process variations to the bitstream. Therefore, PUFs can generate unique challenge-response-pairs (CRPs) as fingerprints for each chip using the uncontrollable manufacturing deviations at the deep submicron level. PUFs can be divided into two categories: weak PUFs and strong PUFs [4]. Strong PUFs, represented by arbiter PUF, have more independent CRPs and are usually employed for authentications. However, strong PUFs are vulnerable to machine learning attacks (MLA) [5].

In this paper, a modeling attack resistant arbiter PUF with a time-variant obfuscation scheme is proposed to resist modeling attack. The major contributions are as follows:

1) The time-variant obfuscation scheme is proposed for the first time, which can be utilized with existing strong PUFs to improve the modeling attack resistance.

2) A 48-stage arbiter PUF with a time-variant scheme (TVO-APUF) is implemented on the Xilinx Spartan-6 FPGA.

3) Experiment on FPGA demonstrates that the proposed TVO-APUF can significantly decrease the prediction rate of modeling attacks.

II. Related Work

Some new or modified structures have been reported to enhance PUF's anti-MLA ability. These structures can be roughly divided into two categories: enhancing the PUF structure's nonlinearity or obfuscating CRPs. It is necessary for enhancing the PUF structure's nonlinearity to increase complexity among PUF's delay elements. However, changes of the PUF's internal element are not generally suitable for implementation on FPGAs. CRP obfuscation aims at hiding the natural mapping relations of CRP by adding pre/post-processing-circuits at challenge input and response output ports, respectively. Therefore, attackers can only collect obfuscated "CRPs", but the real CRPs cannot be accessed, increasing the resistance to machine learning attacks. However, obfuscation modules, especially hash function, usually have significant hardware overhead. Besides, the obfuscation technique is essentially a fixed mapping function that cannot effectively resist advanced MLAs.

Several anti-MLA PUFs with CRP obfuscation have been developed in the literature. Controlled PUF (CPUF) [6] employs random hash functions to encode the input challenge bits. Error correction code (ECC) module is required to ensure response consistency, leading a relatively high hardware overhead. More importantly, the helper data leakage helps attackers launch various physical attacks or reliability-based attacks. Gao et al. proposed a hybrid PUF finite state machine (PUF-FSM) [7] structure without ECC module and helper data to solve the above problems. However, PUF-FSM still requires hash logic, causing extra hardware overhead. Besides, the PUF-FSM-based has been broken by the custom CMA-ES algorithm [8].

978-1-6654-3759-2/21/$31.00 ©2021 IEEE
DOI 10.1109/FPL53798.2021.00018

Authorized licensed use limited to: Zhengzhou University. Downloaded on December 16,2021 at 05:02:19 UTC from IEEE Xplore. Restrictions apply.

Slender PUF, RPUF, OB-PUF, and lockdown technique have been proposed to reduce the hardware overhead. Slender PUF [9] generates a response bitstream using PRNG extension challenge, and then randomly selects a substring of the response bitstream for authentication. A random number generator (RNG) is introduced before a strong PUF to randomize the challenge bits and enhance MLAs' resistance in RPUF [10]. OB-PUF [11] employs an RNG to determine the position and specific values of partial challenge bits. Unfortunately, all of the above PUFs with CRP obfuscation have been broken by advanced MLAs [8], [12]. Lockdown technique [13] sets an upper limit on the amount of accessible CRPs and prevents the attacker from collecting enough CRPs for MLAs. However, this confines the achievable amount of CRPs that can be utilized for authentication.

III. Proposed TVO-APUF Design

A. Design of proposed TVO-APUF

As shown in Fig. 1, the proposed TVO-APUF is generally divided into the following four parts: 1) ***Challenge Generator***; 2) ***APUF Unit***; 3) ***LFSR Unit; 4) Control Unit.***

Based on the above structure, the specific working process of the proposed TVO-APUF design is summarized as follows:

1. The *Challenge Generator* generates (n+4)-bit challenge *C*, where n-bit C_1 is considered the input to the n-stage *LFSR Unit* and the remaining 4-bit challenge C_2 is considered the input to the *Control Unit*.
2. The n-stage *LFSR* receives the n-bit C_1 and considers it as the *LFSR's* initial value, then waits for the shift signal S_L sent by the *Control Unit*.
3. The *Control Unit* receives the remaining 4-bit challenge C_2 and determines the number of times to send the shift signal S_L according to the value of C_2. Considering the *Control Unit* sends the shift signal S_L six times, the *LFSR* should be shifted for six times, and then the ultimate output of the *LFSR* serves as n-bit obfuscated challenge C_O. Finally, when the *Control Unit* finishes sending the S_L, it sends the pulse signal S_A, namely the rising edge transition, to the n-stage *APUF*.
4. The *APUF* generates a 1-bit response *R* once the pulse signal S_A is transmitted through two delay chains from start to the final D flip-flop.

B. Design of proposed time-variable obfuscation scheme

In the proposed design scheme, (n+4)-bit external challenge *C* is divided into two parts. In the first part, n-bit C_1 is utilized as the *LFSR*'s initial value, while in the second part, 4-bit C_2 determines the shift times of the *LFSR* and inputs the *Control Unit*.

The value of 4-bit C_2 determines the shift count of the n-stage *LFSR*. For simplicity, the value of C_2 is chosen equal to the shift count of the *LFSR*. Thus, there are 16 shift count possibilities for 4-bit C_2. A base integer about *Shift Count* is predefined to enhance further the modeling attack resistance of the obfuscation scheme with the *LFSR*. Now, the final *Shift Count* can be formulated as,

$$Shift\ Count = |C_2| + Base \tag{1}$$

where $|C_2|$ indicates the value of 4-bit C_2 from (n+4)-bit *C*,

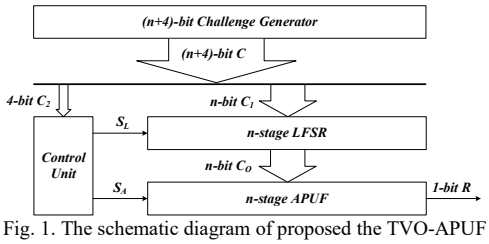


Fig. 1. The schematic diagram of proposed the TVO-APUF

Base is the predefined integer value, and *Shift Count* is the ultimate number of shifting of *LFSR* controlled by shift signal S_L. If the *Base* is equal to 4, the ultimate number of shifting should be increased by 4 to the value of C_2 after each external challenge input. Note that the higher *Base* is, the stronger the modeling attack resistance of the obfuscation scheme is. However, the proposed obfuscation scheme also considers a longer time to generate n-bit obfuscated challenge C_O, which is unacceptable in time-limited applications.

Linear feedback shift register (LFSR) includes a shift register and a linear feedback function employed as a pseudo-random sequence generator. The initial value of the input to LFSR is called the seed, and the bits affecting the LFSR's next state are called taps. Considering different feedback methods, LFSR can be divided into Fibonacci LFSR and Galois LFSR [14]. Due to the reduction in the number of logic gates in the Galois LFSR's feedback loop, its transmission rate is generally faster than the Fibonacci LFSR, the Galois LFSR is chosen in this work.

The proposed obfuscation scheme adopts a time-variant obfuscation mechanism. Several bits are extracted from the external challenge and applied to the mapping function before obfuscating the external challenge. Therefore, the mapping function of the obfuscation module is determined by the specific external challenge in real-time. The "time-variant" obfuscation method employs the changing mapping function to transform the external challenges, which significantly increases the attacker's difficulty to break it by modeling attack. Moreover, the obfuscation logic with LFSR as the core can be easily updated to resist the replay attack.

IV. Proposed TVO-APUF Implementation on FPGA

The proposed TVO-APUF is implemented on Xilinx Spartan-6 XC6SLX9 FPGA using Xilinx's *ISE Design Suite 14.7*, like *FPGA Editor*. Two independent programmable delay lines (PDLs) with low overhead are utilized to introduce a non-path-swapping switch block in implementing arbiter PUF to make the nominal delay of the two delay chains the same and alleviate the system delay bias as possible [15]. A PDL is implemented by a single lookup table (LUT) of FPGA. The arbiter PUF circuit comprises n new non-path-swapping switch units and k tune units (see Fig.2). The upper and lower PDL pairs form a PDL block, which can be divided into two types: switching block and tuning block. The two PDLs of the switching block are input to the same signal, which is the obfuscated challenge bit c_{Oi} ($i = 1, 2, \ldots, n-1, n$), and the two PDLs of the tuning block are input to independent signals, which are the tuning bits t_i^u and t_i^d ($i = 1, 2, \ldots, k$). The switching block is controlled by obfuscated challenge, while the user directly controls the tuning block for adding an appropriate additional delay to the top or

Authorized licensed use limited to: Zhengzhou University. Downloaded on December 16,2021 at 05:02:19 UTC from IEEE Xplore. Restrictions apply.

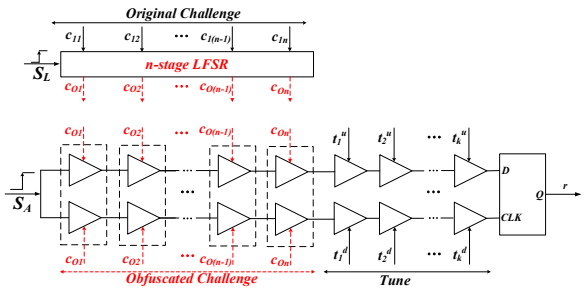


Fig. 2. Arbiter PUF based PDLs

bottom delay chain. The tuning bits are adjusted until the measured uniformity approaches 50%, and the tuning bits can be fixed.

In order to place and route again without changing the underlying arbiter PUF, an unbiased **arbiter PUF hard macro** is designed using the following three steps.

1) **Design of single PDL hard macro.** Hard macros should be made for PDL implementation to ensure that each PDL has the same circuit structure. At first, the following Verilog design *pdlcore.v* is utilized to instantiate a LUT with 6-inputs.

```
// pdlcore. v
module pdlcore (A1, A2, A3, A4, A5, A6, O);
input A1, A2, A3, A4, A5, A6;
output O;
(* LOCK_PINS = "all" *)
LUT6 # (. INIT (64' h 5655555555555555 )) PDL (
.O(O) , . I0 (A1), .I1 (A2), .I2 (A3),
.I3(A4), .I4 (A5), .I5 (A6)) ;
endmodule
```

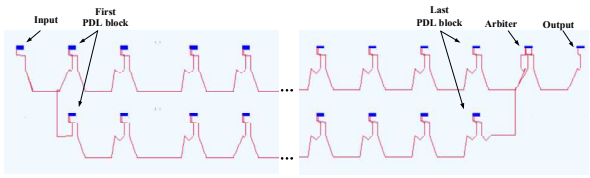
After *Synthesize* and *Implement Design* using *ISE*, a file called *pdlcore.ncd* appears in the file. However, the Verilog design does not implement *O = A1 = F (A1, A2, A3, A4, A5, A6)*, only to leave an optimized 1-input LUT instead of a 6-input LUT. It is necessary to modify the NCD file to implement the *O = A1 = F (A1, A2, A3, A4, A5, A6)* logical function with all input variables. *FPGA Editor* is adopted to modify the NCD file directly. Specifically, the logical expression of LUT should be modified from (2) to (3).

$$O6 = ((\sim A1 * (A2 + (A3 + (\sim A4 + (\sim A5 + \sim A6))))) + (A1 * (\sim A2 * (\sim A3 * (A4 * (A5 * A6)))))) \quad (2)$$

$$O6 = ((\sim A6 * A1) + (\sim A5 * A1) + (\sim A4 * A1) + (A3 * A1) + (\sim A2 * A1) + (A2 * A1)) \quad (3)$$

Hard macro *pdlcorehm.nmc* can be implemented by using *pdlcore.ncd* through the following general steps: Firstly, it is required to unplace all the IOBs, and then delete all the power nets and unplaced components. Secondly, it is required to save the modified NCD file *pdlcore.ncd* as hard macro file *pdlcorehm.nmc*, and then add hard macro external pins. Finally, the hard macros can be instantiated directly by skipping the *Synthesize* and *Place & Route* processes. Thus, the PDLs implemented by hard macro are precisely the same and will not be changed subsequently.

2) **Design of PDL chain hard macro.** A hard macro of a PDL chain should be designed to maintain perfect symmetry between the upper and lower delay chains of the arbiter PUF and further eliminate the nominal deviation caused by place and route. As shown in Fig.2, the PDL hard macro *pdlcorehm.nmc* should be instantiated (n+k) times, and the above method is then followed to implement an (n+k)-stage PDL chain hard macro *pdlchainhm.nmc*.



(a) First PDL block (b) Last PDL block

Fig. 3. Placement and routing of the arbiter PUF hard macro

3) **Design of arbiter PUF hard macro**. It is also necessary to control the routing delay before the first switching block and the routing between the last switching block and the D flip-flop to obtain the same total routing delay for all sections of the two delay chains. Furthermore, arbiter PUF and obfuscation logic are relatively independent in the proposed TVO-APUF design. It is required to *Synthesize* and *Place & Route* again after modifications to obfuscation logic coded by behavioral-level HDL. This leads to unacceptable results influencing the original routing of the arbiter PUF circuit. The best way to solve the above problem is to implement an **arbiter hard macro**. First, the PDL chain hard macro *pdlchainhm.nmc* is instantiated twice. Then, a D flip-flop is instantiated at the end of the PDL chains, and a LUT is instantiated at the start of the PDL chains to counteract the delay bias of the top and bottom paths. The arbiter PUF hard macro *apufhm.nmc*, presented in Fig. 3, can be instantiated as bottom module directly to implement the proposed time-variant obfuscation scheme or other obfuscation scheme without modification.

V. Experiment Results

The proposed TVO-APUF is implemented on 10 Xilinx Spartan-6 XC6SLX9 FPGAs. Python is employed on the PC to process the received experimental data. Specifically, the PC employs the onboard UART interface to transmit the challenge to the FPGA board and return the response. The PC actually replaces the *Challenge Generator* mentioned in Section III to generate random external challenges. For simplicity, we set *Base* to 0 in (1).

A. Resistance to modeling attack

10 thousand CRPs are collected for arbiter PUF and the proposed TVO-APUF on an FPGA. Both circuits consist of 48-stage switching block and 10-stage tuning block. Mainly, logistic regression (LR), support vector machine (SVM), and evolutionary strategy (ES) are utilized to evaluate the mentioned circuits. The data set is divided into a training set (80%) and a test set (20%). Fig. 4 presents the modeling prediction rate for 48-stage arbiter PUF and 48-stage TVO-

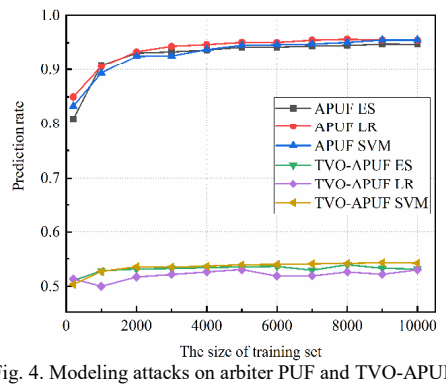


Fig. 4. Modeling attacks on arbiter PUF and TVO-APUF

Authorized licensed use limited to: Zhengzhou University. Downloaded on December 16,2021 at 05:02:19 UTC from IEEE Xplore. Restrictions apply.

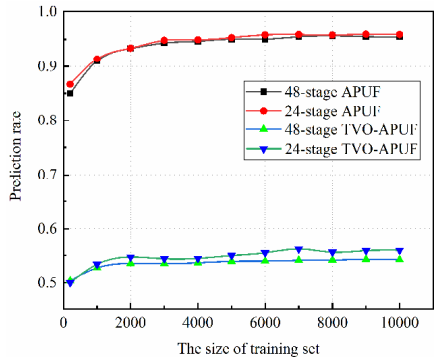


Fig. 5. LR attack on arbiter PUF and TVO-APUF with 24-stage or 48-stage APUF with different amounts of training CRPs.

The modeling attack can effectively predict the arbiter PUF's CRPs according to Fig. 4, even if there are fewer than 100 training data, prediction rate is as high as 0.85, when the training set size is more than 2000, prediction rates are between 0.94 and 0.95, while the three algorithms have almost the same prediction rate. However, since the attacker can only obtain the original external challenge but cannot obtain the internal obfuscated challenge, the three algorithms' prediction rates for TVO-APUF are maintained between 0.5 and 0.53, which is equivalent to random guessing.

Based on the above experiment results, the LR algorithm is utilized to model the 24-stage arbiter PUF and 24-stage TVO-APUF to evaluate further the modeling attack resistance of TVO-APUF under the stage variations. Fig. 5 compares its prediction rate with that of 48-stage arbiter PUF and 48-stage TVO-APUF. The prediction rate of TVO-APUF is much lower than that of arbiter PUF for both 24-stage and 48-stage. Notably, the 24-stage TVO-APUF has a slightly higher prediction rate than the 48-stage TVO-APUF. As a result, the time-variant obfuscation scheme significantly improves the modeling attack resistance regardless of whether the arbiter PUF is 24-stage or 48-stage.

B. Uniformity, Stability, and Uniqueness

In this measurement, we randomly generate 10,000 challenges for TVO-APUF. The average values of uniformity, stability, and uniqueness are shown in TABLE I. The above three performance results are close to the ideal values, indicating that the introduced time-variant obfuscation scheme has little effect on PUF performance.

VI. Conclusion

This paper mainly proposes a TVO-APUF that is resistant to modeling attacks. An LFSR unit is placed before the challenge input of the arbiter PUF to obfuscate the external challenges, making it impossible for an attacker to collect the efficient CRPs for an accurate modeling attack. This paper describes how to implement the proposed TVO-APUF on FPGA, which is also meaningful for implementing other arbiter PUF-based obfuscation schemes. Experimental results on FPGA demonstrate that the proposed TVO-APUF can effectively prevent modeling attacks with a maximum prediction rate of 53%, while the maximum prediction rate is as high as 95% on the arbiter PUF. Moreover, the introduced obfuscation scheme has almost no considerable effect on the uniformity, stability, and uniqueness of the arbiter PUF.

TABLE I
Uniformity, Stability, and Uniqueness

	This Work	Ideal Value
Uniformity	53.2%	50%
Stability	94.2%	100%
Uniqueness	47.1%	50%

However, since PUF has a high requirement for stability in the actual authentication, it should be further considered how to improve the stability. One possible solution is to introduce a voter on the response output and the stable responses can be generated after voting.

References

[1] Abdullah Al Hasib and Abul Ahsan Md. Mahmudul Haque, "A Comparative Study of the Performance and Security Issues of AES and RSA Cryptography," in *Proc. Third International Conference on Convergence and Hybrid Information Technology*, 2008, pp. 505-510.

[2] S. Guo, X. Zhao, F. Zhang, T. Wang, Z. J. Shi, F.-X. Standaert, et al., "Exploiting the Incomplete Diffusion Feature: A Specialized Analytical Side-channel Attack Against the AES and Its Application to Microcontroller Implementations," *IEEE Transactions on Information Forensics and Security*, vol. 9, no. 6, pp. 999-1014, June. 2014.

[3] Y. Gao, S. F. Al-Sarawi, and D. Abbott, "Physical unclonable functions", *Nature Electronics*, vol. 3, no. 2, pp. 81-91, Feb. 2020.

[4] J. Guajardo, S. Kumar, G. Schrijen and P. Tuyls, "FPGA intrinsic PUFs and their use for IP protection," in *Proc. Cryptographic Hardware and Embedded Systems (CHES)*, 2007, pp. 63-80.

[5] U. R Ührmair, F. Sehnke, J. S Ölter, G. Dror, S. Devadas, and J. Ü. Schmidhuber, "Modeling attacks on physical unclonable functions," in *Proc. ACM conference on Computer and communications security (CCS)*, 2010, pp. 237–249.

[6] B. Gassend, M. Van Dijk, D. Clarke, E. Torlak, S. Devadas, and P. Tuyls, "Controlled physical random functions and applications," *ACM Transaction on Information and System Security*, vol. 10, no. 4, pp. 1–22, January. 2008.

[7] Y. Gao, H. Ma, S. F. Al-Sarawi, D. Abbott, and D. C. Ranasinghe, "PUF-FSM: A controlled strong PUF," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 5, pp. 1104–1108, May 2018.

[8] J. Delvaux, "Machine-learning attacks on polyPUFs, OB-PUFs, RPUFs, LHS-PUFs, and PUF–FSMs," *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 8, pp. 2043–2058, Aug. 2019.

[9] M. Majzoobi, M. Rostami, F. Koushanfar, D. S.Wallach, and S. Devadas, "Slender PUF protocol: A lightweight, robust, and secure authentication by substring matching," in *Proc. IEEE Symposium on Security and Privacy Workshops*, 2012, pp. 33–44.

[10] J. Ye, Y. Hu and X. Li, "RPUF: Physical unclonable function with randomized challenge to resist modeling attack," in *Proc. IEEE Asian Hardware-Oriented Security and Trust (AsianHOST)*, 2016, pp. 1-6.

[11] Y. Gao *et al.*, "Obfuscated challenge-response: A secure lightweight authentication mechanism for PUF-based pervasive devices," in *Proc. IEEE International Conference on Pervasive Computing and Communication Workshops (PerCom Workshops)*, 2016, pp. 1-6.

[12] G. T. Becker, "On the pitfalls of using arbiter-PUFs as building blocks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 34, no. 8, pp. 1295–1307, Aug. 2015.

[13] M.-D. Yu, M. Hiller, J. Delvaux, R. Sowell, S. Devadas, and I. Verbauwhede, "A lockdown technique to prevent machine learning on PUFs for lightweight authentication," *IEEE Transactions on Multi-Scale Computing Systems*, vol. 2, no. 3, pp. 146–159, 1 July-Sept. 2016.

[14] M. Goresky and A. M. Klapper, "Fibonacci and Galois representations of feedback-with-carry shift registers," *IEEE Transactions on Information Theory*, vol. 48, no. 11, pp. 2826-2836, Nov. 2002.

[15] M. Majzoobi, F. Koushanfar and S. Devadas, "FPGA PUF using programmable delay lines," in *Proc. IEEE International Workshop on Information Forensics and Security*, 2010, pp. 1-6.

Authorized licensed use limited to: Zhengzhou University. Downloaded on December 16,2021 at 05:02:19 UTC from IEEE Xplore. Restrictions apply.